

Compte-rendu hebdomadaire

Compte-rendu 5

Valentin Noyé

17 février 2025 - 21 février 2025

GitHub : <https://github.com/owilo/Stage>

1 Problématique

Tandis que nous continuons à accorder de l'importance à la recherche et l'implémentation de modèles permettant une translation efficace d'un chiffre de la base de données MNIST vers un autre en maintenant son style, l'une des principales étapes de ce stage concerne la détection et la reconnaissance de traces lors d'une translation dans la mesure pour laquelle nous serions capable de contrôler leur présence.

2 Travail réalisé

2.1 Détection des traces dans la translation

2.1.1 Procédé

Pour reconnaître des traces dans la translation, plusieurs modèles de classifieurs différents ont été employés, et se basent sur une architecture similaire qu'utilisée précédemment. Parmi ces modèles :

1. Un classifieur honnête qui est exactement le même que précédemment, sur nos 10 classes (voir section 2.1.2) ;
2. Un petit modèle de classification binaire sur l'existence de la translation, entraîné sur 2 classes source et une classe de destination, ainsi qu'un grand modèle entraîné sur toutes les classes sources vers toutes les classes de destination (voir section 2.1.3) ;
3. Un petit modèle de reconnaissance de la classe source avec ou sans translation, entraîné sur 2 classes source et une classe de destination, ainsi qu'un grand modèle entraîné sur toutes les classes sources vers toutes les classes de destination (voir section 2.1.4).

Ces quatre derniers modèles sont donc entraînés avec ces spécificités :

- Le petit modèle est utilisé en plus de notre β -VAE. Il est entraîné sur environ 2 400 images de la base de test sur 3 classes c_1 , c_2 et c_d , soit 800 images par classe, afin de laisser 200 images par classe pour nos tests. Parmi ces 800 images, la moitié, soit 400 images de c_1 et c_2 , restent inchangées et l'autre moitié est traduite en c_d . Nous faisons de même avec les 200 autres images. Nous labéllisons alors les images avec leur classe d'origine. Ce modèle emploie la même architecture que notre classifieur simple ;
- Le grand modèle fonctionne tel que le petit modèle mais en partant de toute classe vers toute autre classe de destination. Un nouveau β -VAE est entraîné avec 50% de son dataset d'entraînement afin de garder 40 000 images (4 000 images/classe) — couplé avec le dataset de test — pour effectuer ces tests à grande échelle. Le classifieur est entraîné avec 75% de

ce dataset (3 000 images/classe) tandis que les 1 000 images/classe restantes sont utilisées pour collecter nos données et interpréter les résultats sur de nouvelles images. Ces 3 000 et 1 000 images/classe sont alors divisées en 10 groupes d’images translatées vers d’autres classes de destination, à l’exception de la classe source où les images restent inchangées. Ce modèle emploie la même architecture que notre classifieur simple, mais le nombre de convolutions est doublé.

2.1.2 Classification simple

Cela dit, nous procédons à la simple classification de nos images inchangées et translatées dans la figure 1. Nous retrouvons une précision un peu moins bonne que sur MNIST classique, mais il est possible d’attribuer cela à la nouvelle taille du dataset de l’auto-encodeur et au fait que la translation génère souvent des erreurs de classification.

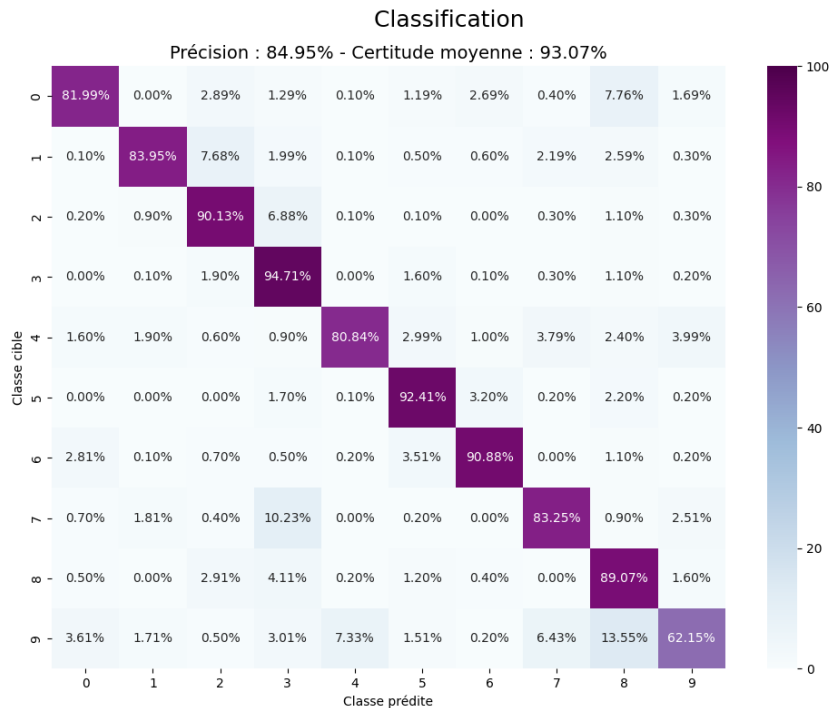


FIGURE 1 – Classification des chiffres inchangés et translatés

Cette bonne classification se répercute sur nos petits modèles (voir figure 2) avec quelques erreurs classiques causées par la translation.

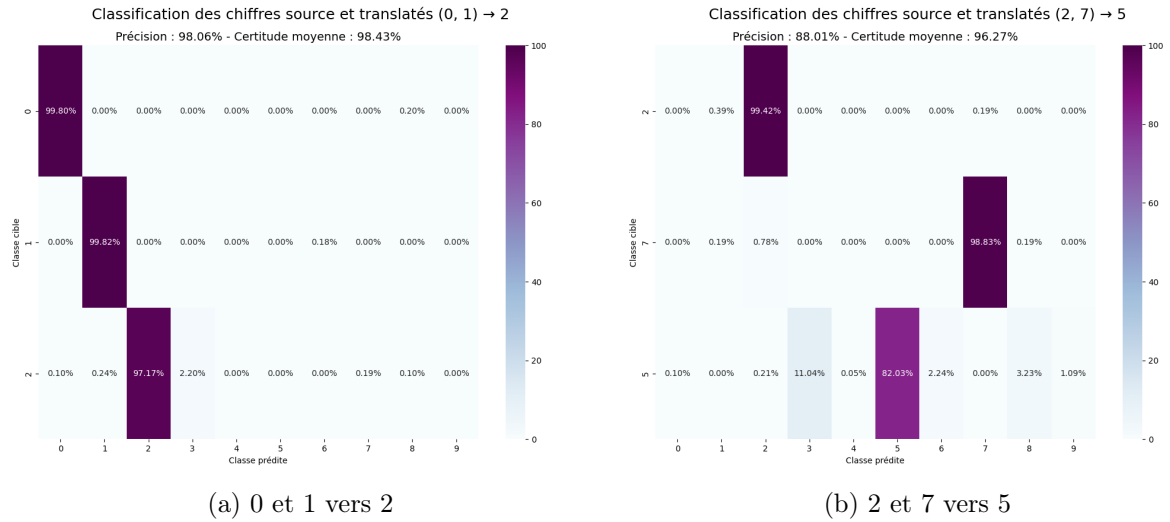


FIGURE 2 – Classification des chiffres inchangés et traduits un modèle réduit

Cette observation se confirme en observant la part d'erreur de classification entre les chiffres traduits et les chiffres inchangés sur la classe de destination dans 3. Nous en déduisons que les chiffres traduits sont en majorité bien classifiés, bien qu'ils augmentent la marge d'erreur de classification.

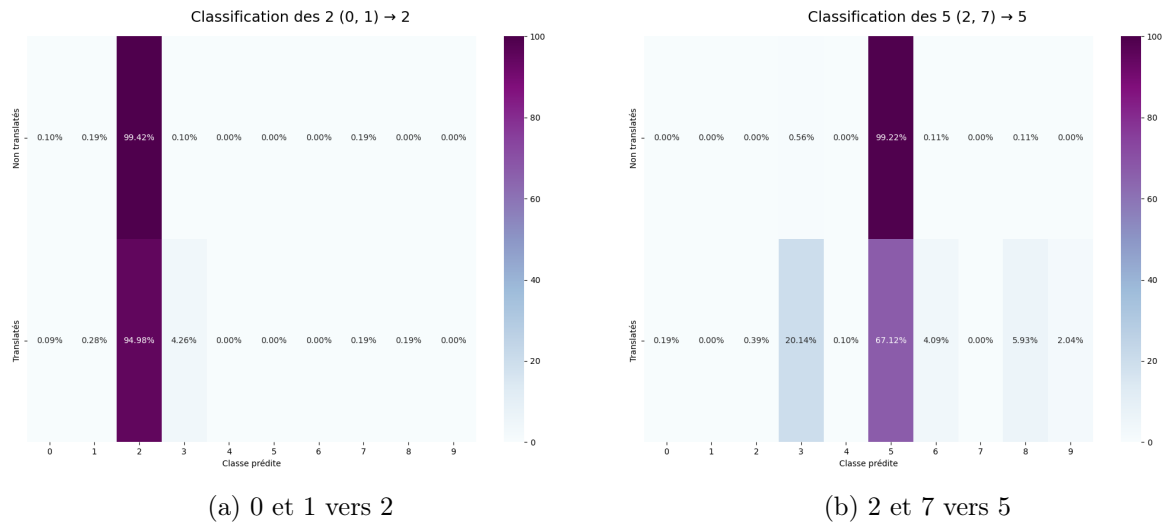


FIGURE 3 – Classification des chiffres de destination inchangés et traduits

2.1.3 Détection de la translation

Le modèle reconnaît avec précision si un chiffre a été traduit ou non, comme le démontre la figure 4.

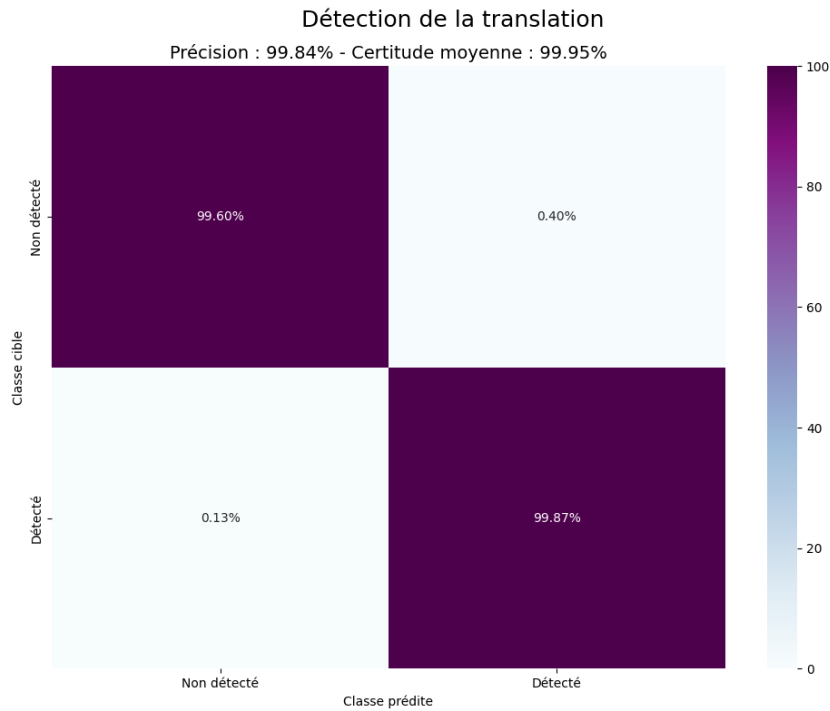


FIGURE 4 – Détection de la translation

2.1.4 Reconnaissance de la classe source

Même si le modèle est très capable de reconnaître la translation, la figure 5 met en évidence le fait que le modèle est limité en sa reconnaissance de la classe d'origine. Malgré ça, ses performances sont plutôt bonnes puisque qu'il reconnaît tout de même la classe source d'un chiffre translaté (ou non) avec 71,75% de précision.

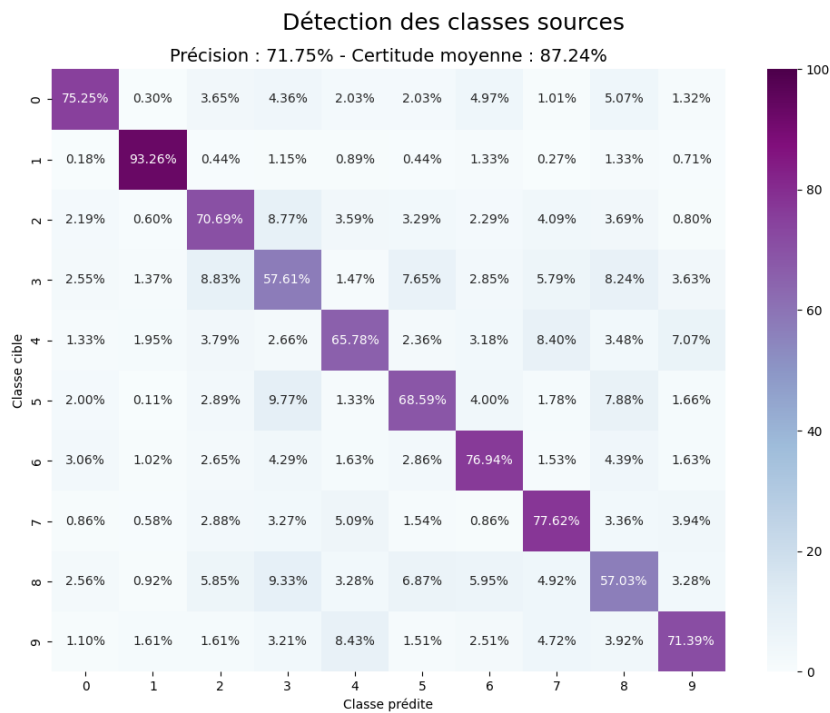


FIGURE 5 – Reconnaissance de la classe source

Cette précision est largement meilleure sur des modèles réduits (voir figure 6).

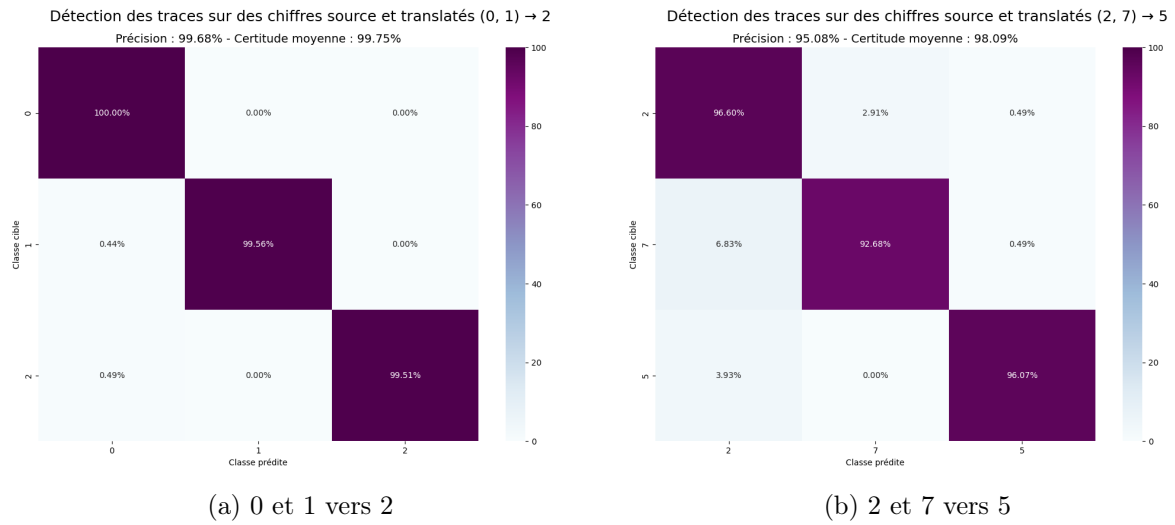


FIGURE 6 – Reconnaissance de la classe source sur des données réduites

Nous voyons à nouveau que cette perte de précision est causée par la reconnaissance de la classe source sur les données translatées, tandis que le modèle classe les chiffres inchangés plus aisément (voir figure 7).

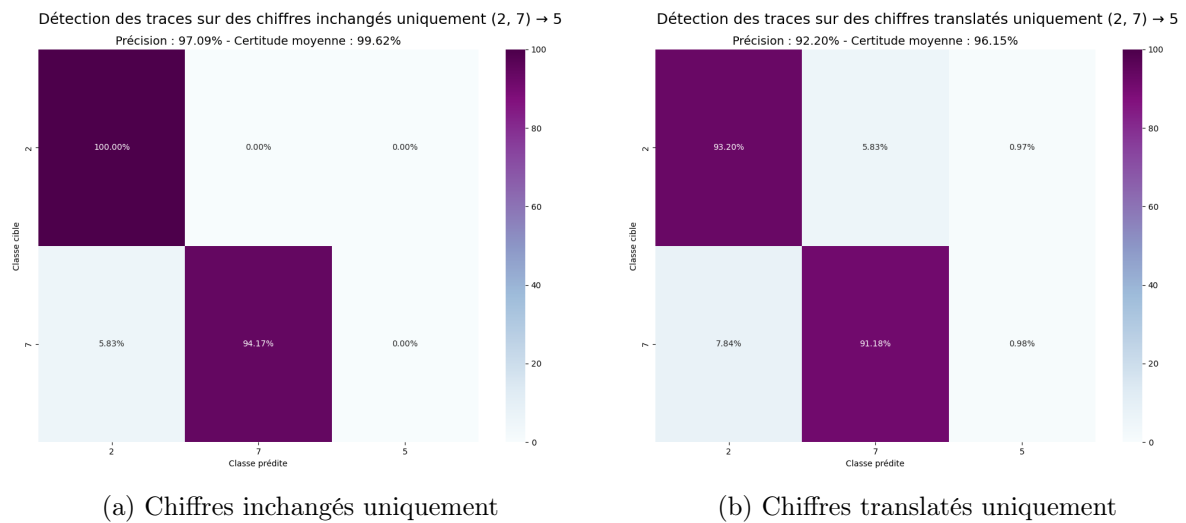


FIGURE 7 – Reconnaissance de la classification de chiffres 2 et 7 vers 5

2.1.5 Sur une photographie d'un chiffre

À présent, nous appliquons cela sur notre photographie du 2. Après seuillage et encodage, nous translatons l'image vers différentes classes sources dans la figure 8. Il est intéressant de remarquer que la classification honnête (en bleu) classe toujours correctement l'image tandis que le modèle de reconnaissance de la classe source (en rouge) définit l'image reconstruite comme un 9. Il en découle ainsi que ce modèle reconnaît la majorité des images translatées comme provenant de la classe source 9.

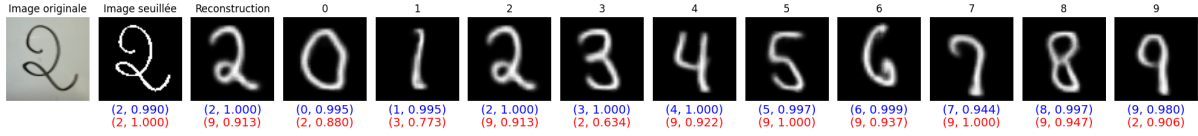


FIGURE 8 – Translation d'un chiffre photographié et reconnaissance de sa classe source

2.1.6 Projections t-SNE des chiffres inchangés et traduits

Nous projetons l'espace de nos vecteurs latents inchangés et traduits en 2D à l'aide d'un t-SNE. Dans la figure 9a, nous observons les 10 clusters des classes, formés par les images inchangées ainsi que traduites vers cesdites classes. Les différentes flèches représentent la translation de notre photographie du 2 vers les autres classes. La figure 9b met en valeur l'origine des points traduits dans chaque cluster de classe représentés par une ellipse. Par exemple, un point jaune dans une ellipse verte représente un 8 traduit en 2. Les ellipses représentent 1,5 écarts-types de la distribution des chiffres de chaque classe.

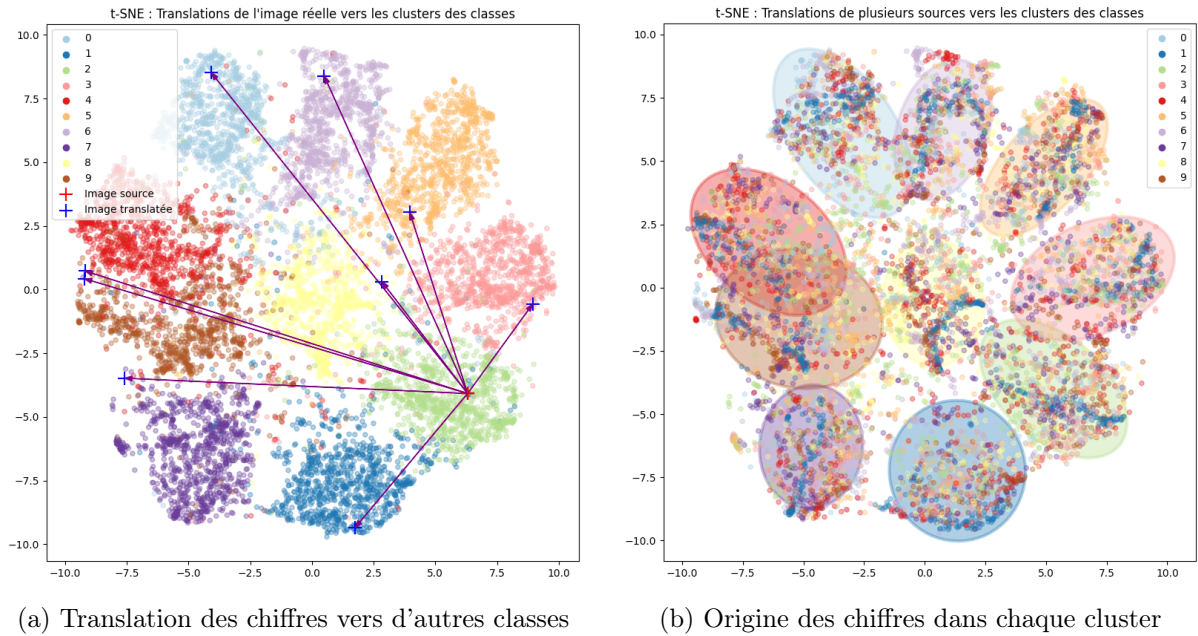


FIGURE 9 – Projections t-SNE de l'espace latent

Cette visualisation peut être menée plus simplement sur un jeu de données plus réduit (voir figure 10). Cela nous permet de visualiser les frontières et les translations plus simplement. La flèche représente la translation de notre 2 photographié vers la classe de destination.

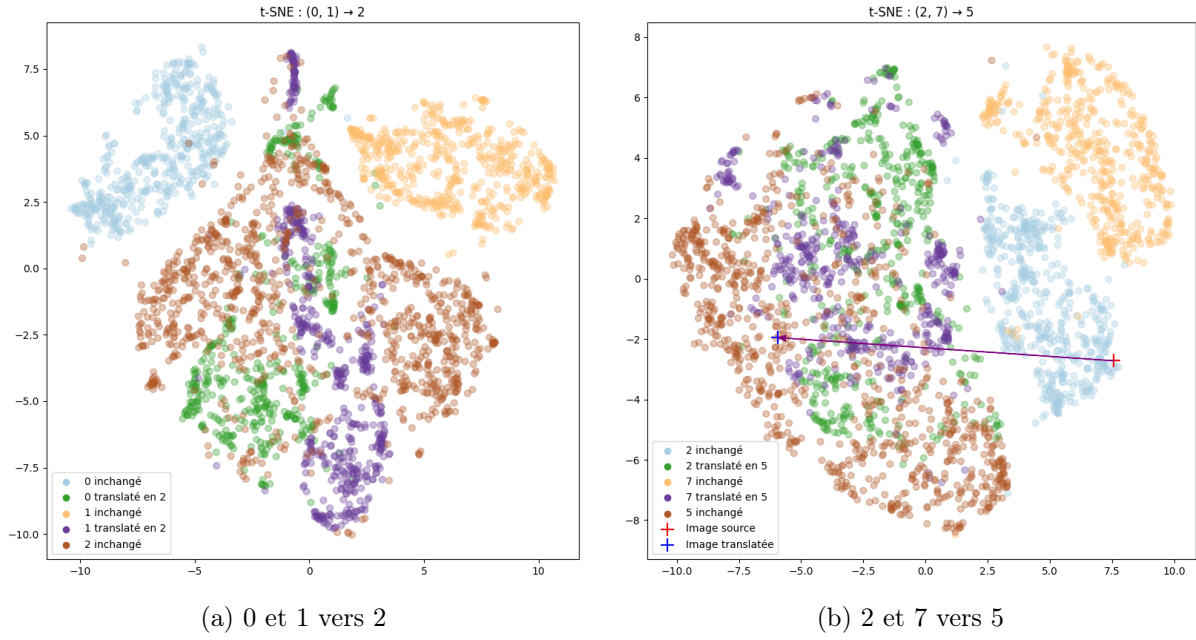


FIGURE 10 – Projections t-SNE de l'espace latent sur un modèle réduit

2.2 Implémentation d'un auto-encodeur variationnel conditionnel

Un VAE conditionnel (CVAE) a été simplement implémenté en concaténant le vecteur aplati suite aux convolutions avec la classe du chiffre encodée sous format catégorique (one-hot), ce qui transforme le vecteur latent en un sous-vecteur continu pour le style, et un sous-vecteur discret de taille 10 pour la classe. Pour traduire un chiffre de sa classe vers une autre, il s'agit de désactiver tous les bits sauf celui de la classe de destination. Le résultat de la translation vers d'autres classes est inscrit dans la figure 11. La première colonne représente le chiffre source.

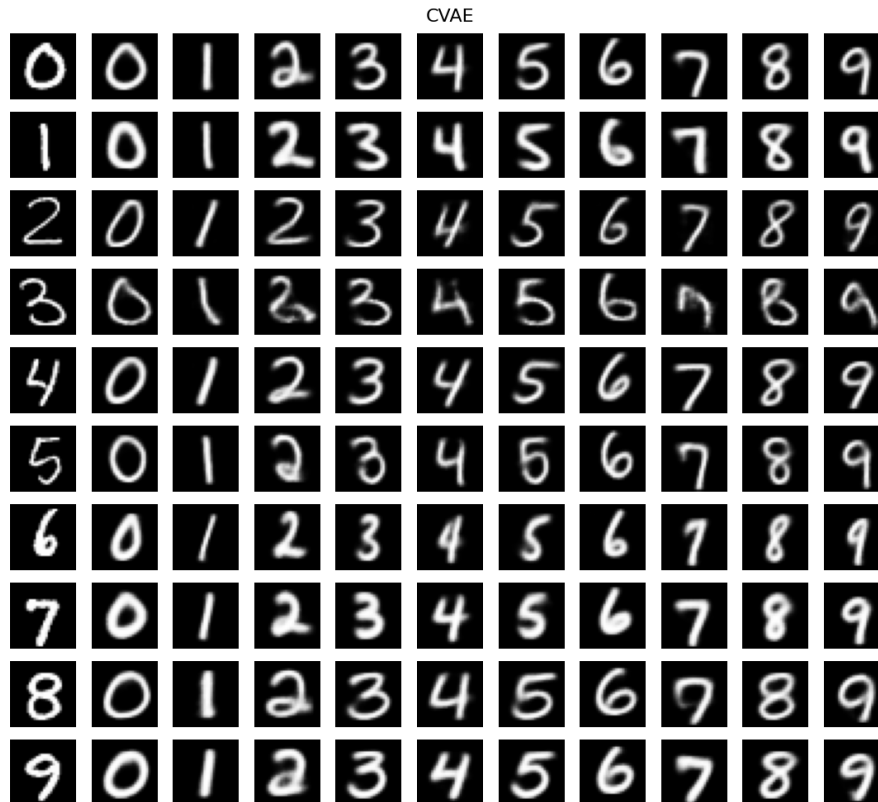


FIGURE 11 – Translation de plusieurs chiffres vers d'autres classes à l'aide du CVAE

3 Travail à effectuer

- Tracer les différentes distributions sur un vecteur latent de taille 16 ;
- Continuer d'explorer les auto-encodeurs semi-supervisés (CVAEs) [1] ou d'autres méthodes similaires ;
- Explorer les méthodes d'empoisonnement afin de tromper le classifieur dans la détection de la classe source.

4 Activités

Réunions :

- Réunion de suivi - Mercredi 19/02/25

Références

- [1] N. Siddharth, Brooks Paige, Jan-Willem van de Meent, Alban Desmaison, Noah D. Goodman, Pushmeet Kohli, Frank Wood, and Philip H. S. Torr. Learning disentangled representations with semi-supervised deep generative models, 2017.